Displays 1: How to set up 7-segment displays

Purpose:

In this tutorial I'm going to show you how to setup and program single digit seven segment displays on an Arduino.

Materials:

Computer, Arduino Uno Board, USB cable, 7 segment display (common anode digital tube), 12 jumper cables, 1 220 - 1000 Ohm resistor, breadboard.

New Components:

Seven segment displays are used in many day-to-day consumer devices like microwave ovens, washing machines, and air conditioners. They are a simple but effective way to display numerical data like time or quantity. Since they are made out of LEDs, they are a low cost option for displaying information.

Seven segment displays come in a wide variety of sizes and colors. Red, blue, and green are the easiest colors to find. Sizes range from small 0.56 inch displays up to large 4 inch and even 6.5 inch displays. Some displays have a single digit, and others have two or four.

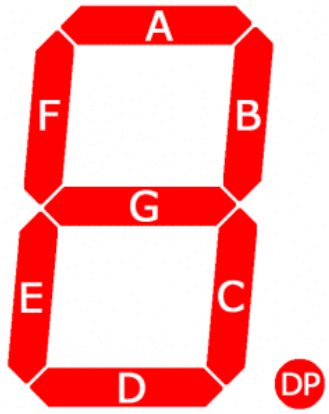


Before we start working with 7 segment displays, we need to understand some of the basics of LEDs and how to control them please complete Activity 1A Blinky before you start.

Procedure:

HOW 7-SEGMENT DISPLAYS WORK

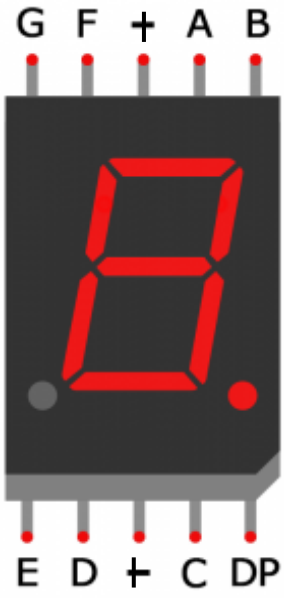
Seven segment displays consist of 7 LEDs, called segments, arranged in the shape of an “8”. Most 7-segment displays actually have 8 segments, with a dot on the right side of the digit that serves as a decimal point. Each segment is named with a letter A to G, and DP for the decimal point:



Each segment on the display can be controlled individually, just like a regular LED.
There are two types of 7-segment displays – common cathode and common anode. **In the SaskCode kits is a common anode.**

COMMON ANODE DISPLAYS

In common anode displays, all of the anodes are connected to Vcc, and individual segments are turned on and off by switching power to the cathodes:



With funding from Canada

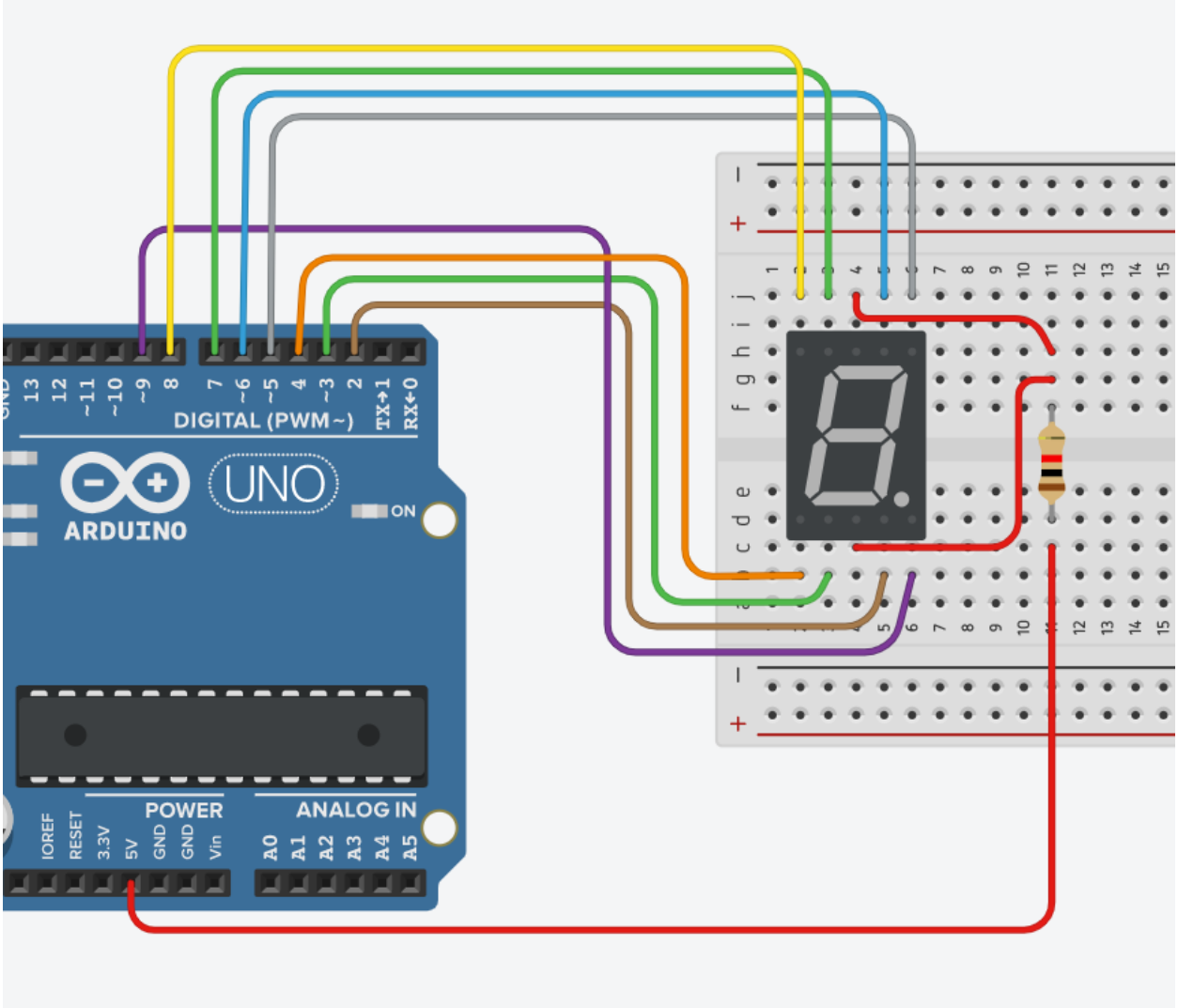
<SaskCode>

SIEC Saskatoon Industry Education Council

Circuit

CONNECTING SINGLE DIGIT DISPLAYS TO THE ARDUINO

Once you have the pin layout figured out, connecting the display to an Arduino is pretty easy. This diagram shows how to connect a single digit 5161AS display (notice the the 220 ohm current limiting resistor connected in series with the common pins):



With funding from
Canada
<SaskCode>
SIEC
Saskatoon Industry Education Council

In the example programs below, the segment pins connect to the Arduino according to this table:

Segment Pin	Arduino Pin
A	6
B	5
C	2
D	3
E	4
F	7
G	8
DP	9

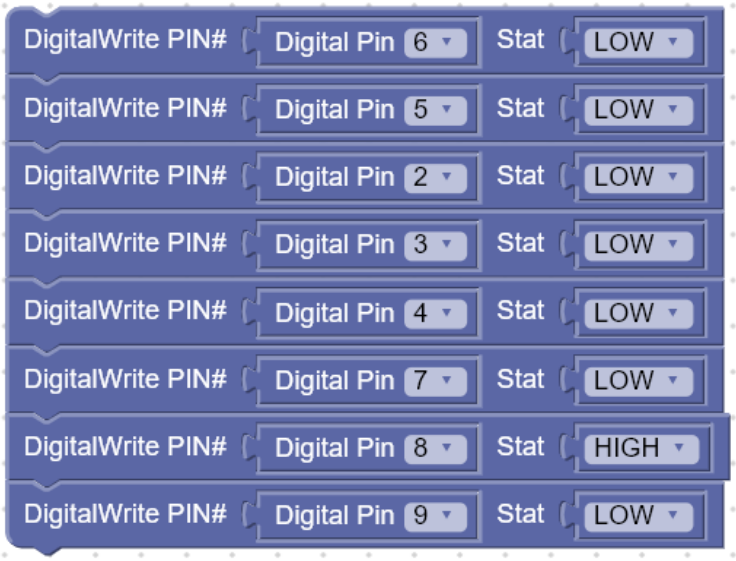
Code

COUNT UP TIMER

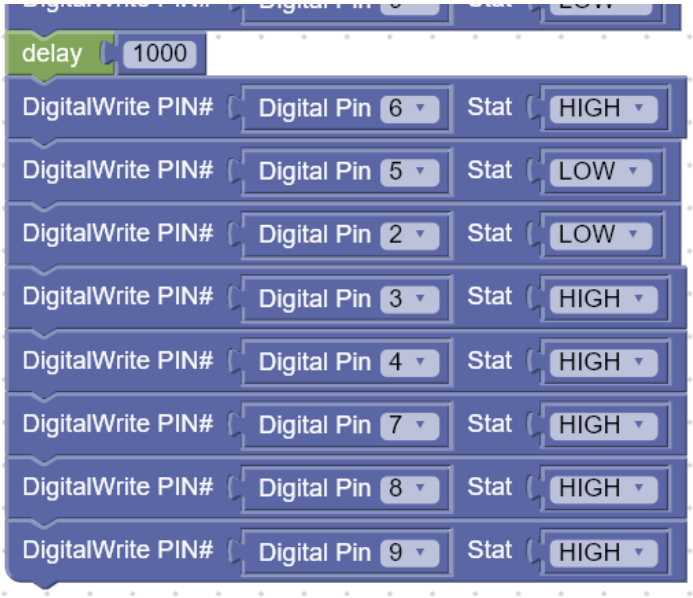
This simple program will count up from zero to 9 and then loop back to the start:

1. You can create your code in the BlocklyDuino then paste it into the Arduino IDE to upload to the ARDUINO.

2. First we need to create the digital blocks to turn the LEDs in the display on. We require 8 blocks for the 8 pins. **Note the "HIGH" would mean the block is off and "LOW" is on.** So the example below is for the number zero.



3. You must create one of these sets of code for each number. A good way to copy complete sets of code is to put them in an if statement and then copy the entire statement. This will allow all 8 blocks to be copied together at once. You can now add a “delay” of 1000 millis to count by seconds, then change the number to one.



4. Continue this process for each number up to 9.

*****SAVE your sketch to your BlocklyDuino Folder that you created*****

You can now copy this by using “ctrl A” then “ctrl C”. Then paste this into the Arduino IDE. Select UPLOAD to program the following sketch onto your board.

-OR-

Copy and paste the following code into your Arduino IDE

```
void setup()
{
  pinMode(6, OUTPUT);
  pinMode(5, OUTPUT);
  pinMode(2, OUTPUT);
  pinMode(3, OUTPUT);
  pinMode(4, OUTPUT);
  pinMode(7, OUTPUT);
  pinMode(8, OUTPUT);
```

```
  pinMode(9, OUTPUT);
}


void loop()
{
  digitalWrite(6, LOW);
  digitalWrite(5, LOW);
  digitalWrite(2, LOW);
  digitalWrite(3, LOW);
  digitalWrite(4, LOW);
  digitalWrite(7, LOW);
  digitalWrite(8, HIGH);
  digitalWrite(9, LOW);
  delay(1000);
  digitalWrite(6, HIGH);
  digitalWrite(5, LOW);
  digitalWrite(2, LOW);
  digitalWrite(3, HIGH);
  digitalWrite(4, HIGH);
  digitalWrite(7, HIGH);
  digitalWrite(8, HIGH);
  digitalWrite(9, HIGH);
  delay(1000);
  digitalWrite(6, LOW);
  digitalWrite(5, LOW);
  digitalWrite(2, HIGH);
  digitalWrite(3, LOW);
  digitalWrite(4, LOW);
  digitalWrite(7, HIGH);
  digitalWrite(8, LOW);
  digitalWrite(9, HIGH);
  delay(1000);
  digitalWrite(6, LOW);
  digitalWrite(5, LOW);
  digitalWrite(2, LOW);
  digitalWrite(3, LOW);
  digitalWrite(4, HIGH);
  digitalWrite(7, HIGH);
  digitalWrite(8, LOW);
  digitalWrite(9, HIGH);
  delay(1000);
  digitalWrite(6, HIGH);
  digitalWrite(5, LOW);
  digitalWrite(2, LOW);
  digitalWrite(3, HIGH);
  digitalWrite(4, HIGH);
  digitalWrite(7, LOW);
  digitalWrite(8, LOW);
  digitalWrite(9, HIGH);
```

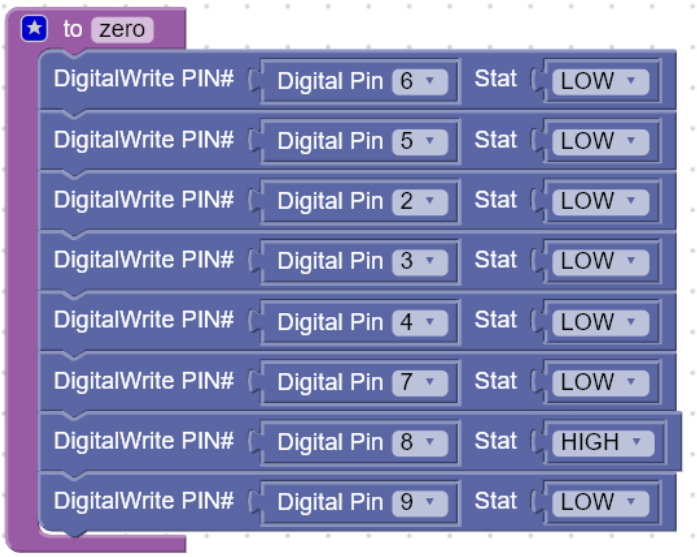
```
delay(1000);
digitalWrite(6, LOW);
digitalWrite(5, HIGH);
digitalWrite(2, LOW);
digitalWrite(3, LOW);
digitalWrite(4, HIGH);
digitalWrite(7, LOW);
digitalWrite(8, LOW);
digitalWrite(9, HIGH);
delay(1000);
digitalWrite(6, HIGH);
digitalWrite(5, HIGH);
digitalWrite(2, LOW);
digitalWrite(3, LOW);
digitalWrite(4, LOW);
digitalWrite(7, LOW);
digitalWrite(8, LOW);
digitalWrite(9, HIGH);
delay(1000);
digitalWrite(6, LOW);
digitalWrite(5, LOW);
digitalWrite(2, LOW);
digitalWrite(3, HIGH);
digitalWrite(4, HIGH);
digitalWrite(7, HIGH);
digitalWrite(8, HIGH);
digitalWrite(9, HIGH);
delay(1000);
digitalWrite(6, LOW);
digitalWrite(5, LOW);
digitalWrite(2, LOW);
digitalWrite(3, LOW);
digitalWrite(4, LOW);
digitalWrite(7, LOW);
digitalWrite(8, LOW);
digitalWrite(9, HIGH);
delay(1000);
digitalWrite(6, LOW);
digitalWrite(5, LOW);
digitalWrite(2, LOW);
digitalWrite(3, HIGH);
digitalWrite(4, HIGH);
digitalWrite(7, LOW);
digitalWrite(8, LOW);
digitalWrite(9, HIGH);
delay(1000);
digitalWrite(6, HIGH);
digitalWrite(5, HIGH);
digitalWrite(2, HIGH);
digitalWrite(3, HIGH);
```

```
  digitalWrite(4, HIGH);
  digitalWrite(7, HIGH);
  digitalWrite(8, HIGH);
  digitalWrite(9, LOW);
  delay(1000);

}
```

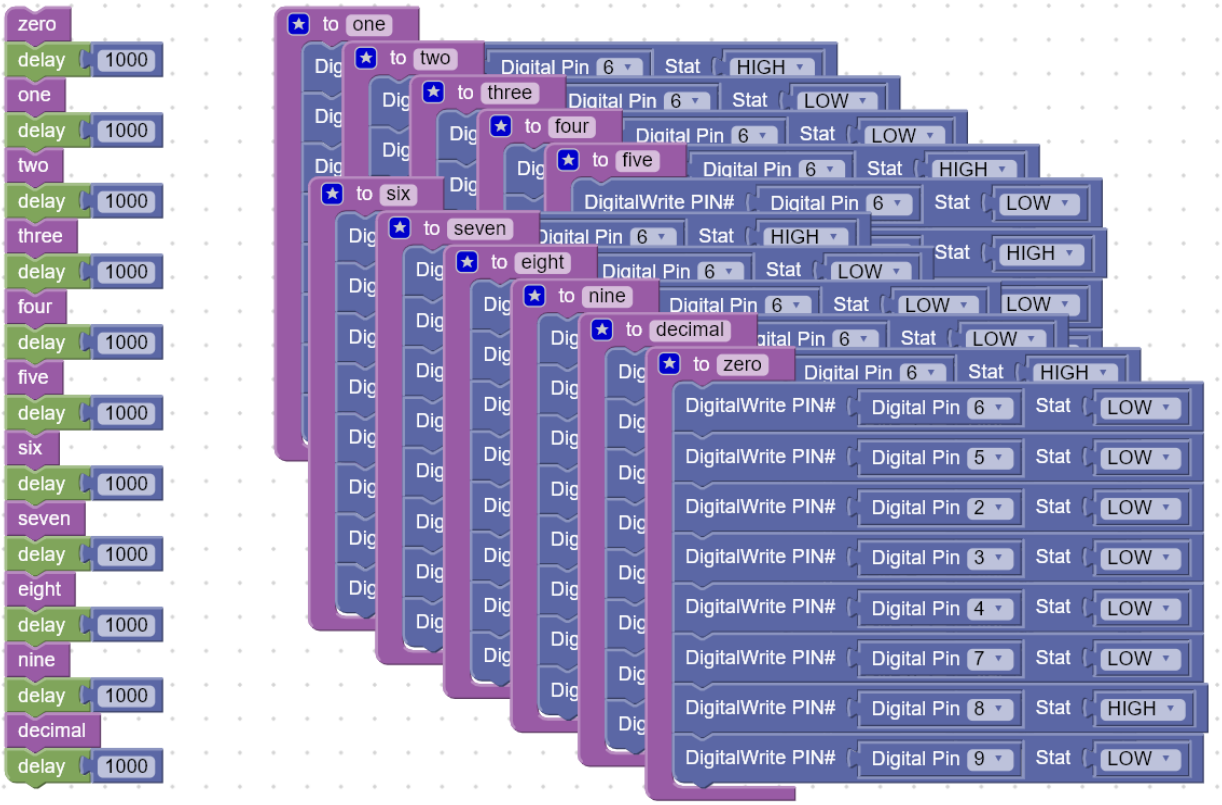
5. Try to rearrange this to create functions for each number. This will allow you to clean up the main program and reuse numbers for multiple displays. For example:





Segmenting code into **functions** allows a programmer to create modular pieces of code that perform a defined task and then return to the area of code from which the **function** was

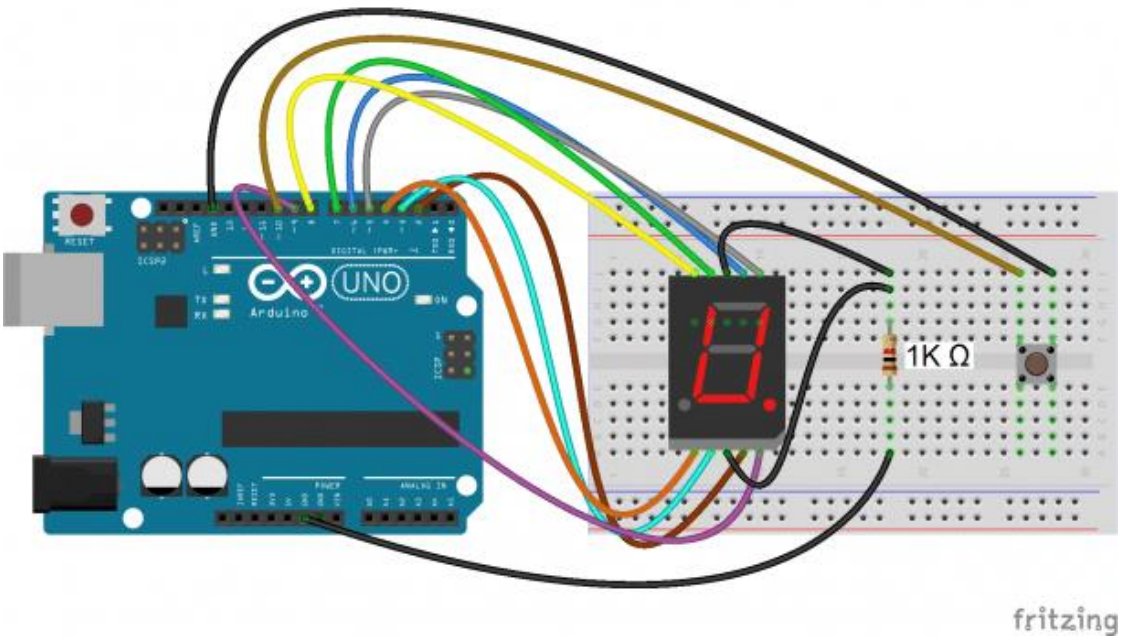
"called". ... Standardizing code fragments into **functions** has several advantages: **Functions** help the programmer stay organized.



6. ROLLING DICE

This example consists of a push button and a single 7 segment display. Every time the push button is pressed and held, the display loops through numbers 0-9 rapidly. Once the button is released, the display continues to loop for a period of time almost equal to the time the button was pressed, and then displays a number along with the decimal point to indicate the new number.

7. Build the circuit as shown below



8. Create a program to generate a number between 1 and 6 when the button is pressed. You will need to use the Random command under the Math menu in BlocklyDuino and create an integer variable.



9. You will now need to call forward the functions for 1 to 6 based on the number generator.

10. Show the result to you teacher!

Activity 4F: How to set up 7-segment displays

1. What are some applications of this kind of counter?

2. Where are some examples of where the random generator could be used?

3. **THE CODE**

    a. Why is it good practice to add functions?

    b. How could you reuse this code? Blockly will not allow you to paste between 2 windows.

Extension Challenges:

4. Could you create this code with no delays? (Hint: See Activity 1F: Blink without delay)

5. Change your code to count down. How about more quickly?

6. How could this be integrated into your stoplight as a countdown for walkers?

With funding from Canada

<SaskCode>

SIEC Saskatoon Industry Education Council